Checkpoint 4 — Tabela de Símbolos e Passada de Identificação

Parte 1 — Desenho do ambiente e decisões de escopo

Maria Rita Vieira Souza- 12321BCC024, Ana Clara Morais de Carvalho- 12311BCC048

Link para o Github do projeto: https://github.com/mariarvsouza/GBC071-Construcao-de-Compiladores

1 Ambiente em cada ponto de um programa

Programa exemplo:

```
1  number g = 100;
2
3  number soma(number x) {
4    number a = 10;
5    for (number i=0; i<x; i=i+1) {
6      number a = 20;
7      number b = a+i+g;
8    }
9    number c = a+g;
10   return c;
11 }
```

Ambiente depois de cada declaração e de cada fechamento de bloco:

Linha	**Ambiente (interno :: externo)**
1 number g = 100;	$\{g@1\}$
3 number soma(number x) {	$\{x@3\} :: \{g@1, soma@3\}$
4 number a = 10;	$\{a@4, x@3\} :: \{g@1, soma@3\}$
5 for (number i=0; i<x; i=i+1) {	$\{i@5\} :: \{a@4, x@3\} :: \{g@1, soma@3\}$
6 number a = 20;	$\{a@6, i@5\} :: \{a@4, x@3\} :: \{g@1, soma@3\}$
7 number b = a+i+g;	$\{b@7, a@6, i@5\} :: \{a@4, x@3\} :: \{g@1, soma@3\}$
8 }	$\{a@4, x@3\} :: \{g@1, soma@3\}$
9 number c = a+g;	$\{c@9, a@4, x@3\} :: \{g@1, soma@3\}$
11 }	$\{g@1, soma@3\}$

Para cada uso, a declaração a que ele resolve:

Uso	**Linha**	**Resolve para**
x	5 for (number i=0; i<x; i=i+1)	x@3
a	7 number b = a+i+g;	a@6
i	7 number b = a+i+g;	i@5
g	7 number b = a+i+g;	g@1
a	9 number c = a+g;	a@4
g	9 number c = a+g;	g@1
c	10 return c;	c@9

2 Symbol

2.1 Campos do Symbol

O Symbol guarda o nome do identificador (name) para ser usado no declare, lookup e nas mensagens de erro; uma variável (kind) para distinguir se a entidade é uma variável (VAR), parâmetro (PARAM) ou função (FUNC), a qual é usada para impedir chamadas de variáveis como funções ou atribuições como funções; um ponteiro direto para o nó de declaração na AST (Node decl) usado para recuperar a linha e coluna da declaração; o nível de profundidade (level) do escopo do símbolo para identificar redeclarações; e o tipo (type) e o deslocamento (offset), reservados para as as próximas etapas.

Representação da classe Symbol:

```
public class Symbol {
    public final String name;
    public final Kind kind;
    public final Node decl;
    public final int level;

    public Type type;
    public int offset;

    public Symbol(String name, Kind kind, Node decl, int level) {
        this.name = name;
        this.kind = kind;
        this.decl = decl;
        this.level = level;
        this.type = null;
        this.offset = -1;
    }
}
```

2.2 Implementação da API SYmbolTable

Adotou-se a pilha de mapas (StackSymbolTable, implementada com Deque<Map<String, Symbol>>), em vez de escopos encadeados ou tabela única com registro. Ela representa diretamente cada escopo ativo, permite sombreamento e restaura a declaração externa ao desempilhar um mapa. O Visitor usa a interface SymbolTable, e não depende da classe concreta, salvo na criação da tabela.

```
interface SymbolTable {
  void openScope();
  void closeScope();
  boolean declare(Symbol s);
  Symbol lookupLocal(String name);
  Symbol lookup(String name);
  int level();
}
```

Na implementação, a tabela inicia com o global aberto (level()==0); openScope() empilha um mapa; closeScope() desempilha o escopo atual; e tentar fechar o global gera IllegalStateException com mensagem explicativa. O declare retorna false para nome já presente no escopo atual e conserva a primeira declaração e o lookupLocal procura apenas no topo, enquanto lookup busca de dentro para fora; ambos retornam null se não houver símbolo.

3 As cinco decisões de projeto

3.1 Sombreamento e Redeclaração

A escolha para o trabalho foi permitir o sombreamento de nomes externos por internos, mas proibir a redeclaração de um mesmo nome no mesmo escopo, ao invés de proibir o sombremaneto ou permitir redeclaração no mesmo bloco. Essa escolha foi feita por preferência das integrantes do grupo por essas características em linguagens de programação, como o C. Permitir redeclarações no mesmo bloco poderia tornar o código confuso, e o sombremaneto é útil para reutilizar nomes de variáveis.

Programa de teste:

```
number a = 1;
number teste(number b) {
  number a = 2;
  return a+b;
}
number c = 3;
number c = 4;
```

Nesse teste, o sombreamento da variável a é válido, mas a redeclaração de c gera erro.

3.2 Declarar antes de usar

A escolha para o trabalho foi impedir o uso de uma variável antes de sua declaração, então é obrigatório declarar antes de usar. Essa escolha foi feita para permitir a identificação em uma travessia linear e tornar o projeto mais simples.

Programa de teste:

```
x = 10;
number x = 5;
```

Nesse teste, o uso de x antes da declaração gera um erro.

3.3 Funções mutualmente recursivas

A escolha para o trabalho foi impedir o uso de funções mutualmente recursivas, apesar de permitir recrusão direta, já que exigimos a declaração prévia, então uma função só pode chamar funções declaradas acima dela, ao invés de ter que realizar duas passadas na AST. Essa escolha foi feita para permitir uma passagem linear na AST e manter o projeto simples.

Programa de teste:

```
number f() { g(); return 1; }
number g() { f(); return 1; }
```

Nesse teste, o uso de g dentro da função f gera um erro porque g ainda não foi declarado.

Programa de teste 2:

```
number contagem(number n){
  if(n>0) {
    return contagem(n-1);
  }
  return 0;
}
```

Esse teste é aceito, já que a identificação verifica a existência dos nomes.

3.4 Parâmetros e corpo

A escolha para o trabalho foi manter os parâemtros da função e as variáveis declaradas no corpo da função no mesmo escopo, ao invés de o corpo da função abrir um escopo isolado. Essa escolha foi feita porque acreditamos que o corpo da função e seus

parâmetros tem uma única região semântica e não faria sentido considerar eles como escopos separados.

Programa de teste:

```
number soma(number a) {
  number a = 10;
  return a+1;
}
```

Nesse teste, a declaração de a no corpo da função gera um erro de redeclaração no mesmo escopo.

3.5 Espaço de nomes

A escolha para o trabalho foi manter as variáveis e funções em um espaço de nomes compartilhado e único, ao invés de manter espaços de nomes separados para variáveis e funções. Essa escolha foi feita para simplificar a tabela de símbolos.

Programa de teste:

```
char x = 'a';
number x(number y){
    return y;
}
```

Nesse teste, a declaração da função como "x" gera um erro, já que "x" já foi declarado como uma variável char no espaço de nomes.

4 Passada de identificação: integração com a AST

A nova classe VisitorIdentificacao implementa Visitor<Void> e percorre apenas árvores produzidas por Parser. ParseResult quando isSucesso() é verdadeiro. Além disso, a visita acumula erros sem interromper o processamento.

- Programa: visita os comandos sequencialmente, usando o escopo global existente.
- Função: registra o nome no escopo externo antes de abrir o escopo de parâmetros e corpo; isso viabiliza recursão direta.
- Bloco comum: abre um escopo, percorre os comandos na ordem e fecha o escopo; usa finally para garantir o fechamento.
- Laço for: abre um escopo para a inicialização/controle e visita seu bloco, que abre outro escopo.

- Declarações: criam Symbol(name, kind, decl, level) e utilizam declare; se falhar, registram a declaração anterior sem sobrescrevê-la.
- Usos: consultam lookup. Em IdExprNode, o campo decl recebe o Symbol encontrado ou permanece null quando não declarado.
- Usos armazenados como texto no CP3: AssignNode.varID e CallFuncExprN-ode.func são resolvidos pelo Visitor e associados aos seus nós através de um mapa externo (getResolucao(Node)). Assim, não se altera a estrutura dessas duas classes.

Formato dos erros:

```
linha 2: 'a' já declarado neste escopo (linha 1)
linha 3: 'b' não declarado
```

A implementação atual faz resolução de nomes. O campo kind é registrado, mas a validação de se uma chamada é realmente uma função e se uma atribuição tem destino permitido deverá ser incorporada na fase semântica correspondente.

5 Suíte de testes e resultados esperados

O pacote de testes do CP4 contém test/programas/, test/esperado/ e test/esperado_cp3/.

5.1 Tabela isolada

O SymbolTableTest.java testa declaração/busca, redeclaração sem substituição, sombreamento e restauração após closeScope, escopo local, nome inexistente, profundidade (level) e comportamento documentado ao tentar fechar o global.

5.2 Programas de identificação

- **sombreamento_valido.txt**: Usos resolvem declaração interna ou externa conforme o escopo.
- **for_valido.txt**: Inicialização, condição, incremento e corpo do for.
- **recursao_direta.txt**: A função encontra seu próprio nome e seu parâmetro.
- **redeclaracao.txt**: Nome repetido no mesmo bloco gera erro; a primeira declaração fica.
- **nome_nao_declarado.txt**: Uso de identificador inexistente gera erro.
- **declarar_antes.txt**: Referência anterior à declaração é rejeitada.
- **recursao_mutua.txt**: Chamada antecipada à segunda função é rejeitada.

- **parametro_corpo.txt**: Parâmetro e variável local não podem repetir o nome no mesmo escopo.
- **espaco_nomes.txt**: Variável e função com mesmo nome conflitam.
- **for_escopos.txt**: Sombreamento interno e uso de nome após fechamento do bloco.
- **multiplos_erros.txt**: Diversos erros são acumulados em ordem, sem parar no primeiro.

O teste multiplos_erros.txt, por exemplo, possui a seguinte saída esperada:

```
linha 2: 'a' já declarado neste escopo (linha 1)
linha 3: 'b' não declarado
linha 3: 'c' não declarado
linha 4: 'e' não declarado
```

O relatório de vínculos usa linha N: 'nome' $->$ KIND@M, sendo M a linha da declaração; apresenta os erros primeiro e depois os vínculos na ordem de visita à AST. Os arquivos de referência estão em test/esperado/.